UNIVERSIDADE PRESBITERIANA MACKENZIE
Faculdade de Computação e Informática
Prof. Dr. Ivan Carlos Alcântara de Oliveira
Disciplina: Teoría dos Grafos

Implementação

AGCM- Algoritmo de Kruskal ou Pim (Grupo do Projeto)

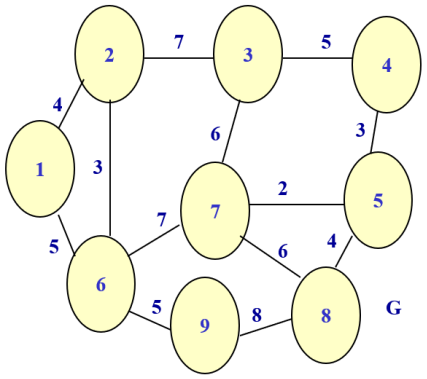
Nome do(a) aluno(a)	RA
Mateus Ribeiro Cerqueira	10443901
Pedro Henrique Carvalho Pereira	10418861

Link do GitHub:
https://github.com/M4t3u5534/implementacao-prim.git

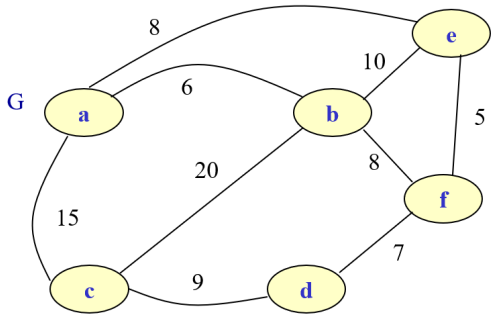
A partir dos algoritmos de Kruskal e Prim estudados em aula sobre Árvore Geradora de Custo Mínimo, escolher um deles e fazer a implementação na linguagem de programação selecionada para o desenvolvimento do seu projeto. Para isso, utilize a classe grafo com matriz de adjacência ou lista de adjacência disponibilizada em aula.

Realizar pelo menos dois testes, sendo: 1) grafo do material da aula; e 2) grafo da atividade solicitada anteriormente, conforme apresentado a seguir.

Grafos a serem utilizados para testes (Necessariamente)



Grafo do Material da Aula



Grafo da Atividade Solicitada Anteriormente

Os códigos-fontes desenvolvidos para esta atividade devem OBRIGATORIAMENTE ser baseados no pseudocódigo do material da aula. Caso não seja feito desta forma, a atividade será zerada.

Colocá-los no relatório a seguir. **Em uma seção final do relatório, denominada Apêndice, inclua o código fonte para o algoritmo desenvolvido, devidamente documentado**.

Os códigos fontes desenvolvidos e o relatório devem ser NECESSARIAMENTE encaminhados pelo Ambiente Virtual.

UNIVERSIDADE PRESBITERIANA MACKENZIE
Faculdade de Computação e Informática
Prof. Dr. Ivan Carlos Alcântara de Oliveira
Disciplina: Teoría dos Grafos

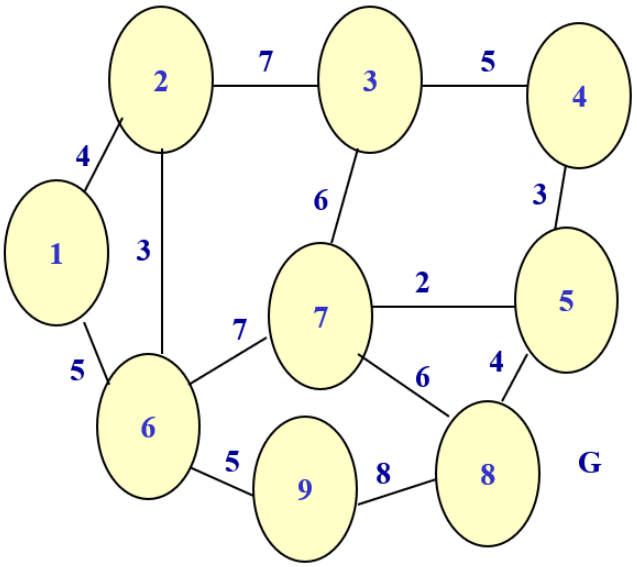
Relatório

DEFINIÇÕES

O algoritmo escolhido para implementação foi o algoritmo de Prim.

Para a execução os grafos foram definidos no código Main.java, a partir das representações indicadas no descritivo da atividade. Consta abaixo, os grafos e suas definições em código:

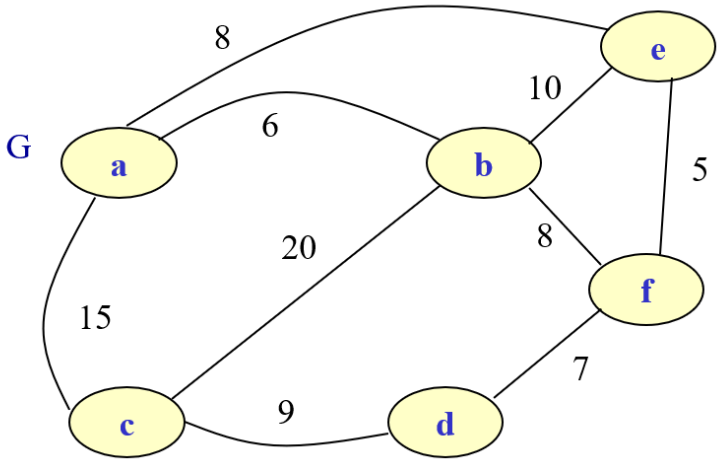
Grafo 1 - Material da Aula




```java
// Grafo 1 (material da aula): vértices 1..9 -> índices 0..8
static TGrafo grafoAula() {
    TGrafo g = new TGrafo(9);
    int[][] arestas = {
        {1,2,4}, {1,6,5}, {2,3,7}, {2,6,3}, {3,4,5}, {3,7,6}, {4,5,3},
        {5,7,2}, {5,8,4}, {6,7,7}, {6,9,5}, {7,8,6}, {8,9,8}
    };
    for (int[] a : arestas) g.insereA(a[0] - 1, a[1] - 1, a[2]);
    return g;
}
```

UNIVERSIDADE PRESBITERIANA MACKENZIE
Faculdade de Computação e Informática
Prof. Dr. Ivan Carlos Alcântara de Oliveira
Disciplina: Teoría dos Grafos

Grafo 2 - Atividade Solicitada Anteriormente




```java
// Grafo 2 (atividade anterior): a..f -> índices 0..5
static TGrafo grafoAtividade() {
    TGrafo g = new TGrafo(6);
    int a = 0, b = 1, c = 2, d = 3, e = 4, f = 5;
    g.insereA(a, e, 8);
    g.insereA(a, b, 6);
    g.insereA(a, c, 15);
    g.insereA(b, e, 10);
    g.insereA(b, f, 8);
    g.insereA(b, c, 20);
    g.insereA(e, f, 5);
    g.insereA(f, d, 7);
    g.insereA(c, d, 9);
    return g;
}
```

ALGORITMO DE PRIM

O objetivo do algoritmo de Prim é construir uma Árvore Geradora Mínima, ou seja, conectar os vértices de um grafo com o menor custo total. No código implementado, usamos os pseudocódigos dos slides, seguindo o fluxo de iniciar a árvore com um único vértice e, a cada etapa, escolher a aresta de menor peso que liga um vértice já pertencente à árvore a outro que ainda não foi inserido nela.

Início <dados: Grafo G = (V, E) >
valor ← ∞; // auxiliar na troca da melhor aresta
custo ← 0; // armazena o custo total da árvore
T ← { 1 }; // Começa a montar a árvore a partir do vértice 1
E' ← ∅; // A árvore não possui nenhuma aresta

No código desenvolvido, essa inicialização foi feita a partir da chamada inicial do Prim

```
public int[] Prim(int origem) {
    Set<Integer> conect = new TreeSet<Integer>();

    int[] ant = new int[n];
    Arrays.fill(ant, -1);

    // INICIALIZAÇÃO
    conect.add(origem);

    PrimRecursivo(conect, ant);

    return ant;
}
```

O conjunto conect possui a mesma função do conjunto T do algoritmo dado em sala, armazenando os vértices que já fazem parte da árvore.

A única diferença é que foi feita uma implementação mais genérica, onde não necessariamente precisa-se iniciar pelo vértice 1, como no algoritmo passado em aula, mas por qualquer vértice inicial através do parâmetro origem.

Após a inicialização do conjunto T, é feita uma chamada ao PRIM(T). No código, essa lógica aparece em:

```
            PrimRecursivo(conect, ant);
```

A recursão começa verificando se todos os vértices já foram inseridos na árvore e em seguida, são inicializadas as variáveis utilizadas para armazenar a melhor aresta encontrada naquela chamada:

```
        private void PrimRecursivo(Set<Integer> conect, int[] ant) {

            if (conect.size() == n) return;

            int r = -1;
            int origemR = -1;
            int p = INF;
```

A variável ‘p’ corresponde ao ‘valor’ do pseudocódigo, armazenando o menor peso encontrado. Já ‘origemR’ corresponde ao vértice interno da árvore, enquanto ‘r’ corresponde ao vértice externo que será adicionado.

A busca pela menor aresta é realizada no trecho abaixo:

O primeiro laço percorre os vértices que já pertencem à árvore. Já o segundo laço percorre todos os vértices do grafo.

```
for (int i : conect) {

    for (int j = 0; j < n; j++) {
```

A condicional abaixo faz com que os vértices que já estão na árvore sejam ignorados. Dessa forma, apenas os vértices externos são analisados, representando o conjunto V - T.

```
if (conect.contains(j)) continue;
```

Já a condição abaixo fica responsável por selecionar a menor aresta:

```
if (adj[i][j] != INF && adj[i][j] < p) {
    r = j;
    origemR = i;
    p = adj[i][j];
}
}
}
```

Esse trecho segue a lógica do algoritmo dado em aula, que percorre cada vértice pertencente a T e compara com os vértices de V - T, procurando a aresta de menor peso.

para todo $k \in T$ ***faça*** / / para todo vértice k pertencente a árvore T
para todo $i \in V - T$ ***faça*** / / procura a aresta menor aresta em V-T
/ / Se alguma aresta é menor do que a encontrada até o momento
se $v_{ki} <$ valor **então**
/ / atualiza o valor, o vértice interno a árvore (vint) e externo (vext)
valor $\leftarrow v_{ki}$; vint $\leftarrow$ k; vext $\leftarrow$ i;
fim-se
fim-para
fim-para / / atualiza o custo, insere o novo vértice vext em T

Continuando, temos:

```
        // sem aresta válida = grafo desconexo
        if (r == -1) return;

        ant[r] = origemR;
        conect.add(r);

        PrimRecursivo(conect, ant);
    }
}
```

Após a busca, o código verifica se nenhuma aresta válida foi encontrada (esse caso ocorre quando não existe uma conexão entre os vértices já inseridos na árvore e os vértices restantes), ou seja, quando o grafo é desconexo a partir da origem escolhida.

```
            // sem aresta válida = grafo desconexo
            if (r == -1) return;
```

Quando uma aresta válida é encontrada, ela é registrada por meio do vetor ant que corresponde ao conjunto E' do algoritmo passado em aula. Assim, se ant[r] = origemR, significa que a aresta entre origemR e r foi selecionada para a árvore geradora.

```
            ant[r] = origemR;
```

Em seguida, o novo vértice é adicionado ao conjunto dos vértices conectados: conect.add(r). Essa operação corresponde à inclusão de vext em t no pseudocódigo dado.

E por fim, temos a chamada recursiva:

```
        PrimRecursivo(conect, ant);
```

UNIVERSIDADE PRESBITERIANA MACKENZIE
Faculdade de Computação e Informática
Prof. Dr. Ivan Carlos Alcântara de Oliveira
Disciplina: Teoría dos Grafos

EXECUÇÃO

```
=== GRAFO 1 (Material da Aula) ===
n: 9
m: 13

Adj[0,0]= - Adj[0,1]= 4 Adj[0,2]= - Adj[0,3]= - Adj[0,4]= - Adj[0,5]= 5 Adj[0,6]= - Adj[0,7]= - Adj[0,8]= -
Adj[1,0]= 4 Adj[1,1]= - Adj[1,2]= 7 Adj[1,3]= - Adj[1,4]= - Adj[1,5]= 3 Adj[1,6]= - Adj[1,7]= - Adj[1,8]= -
Adj[2,0]= - Adj[2,1]= 7 Adj[2,2]= - Adj[2,3]= 5 Adj[2,4]= - Adj[2,5]= - Adj[2,6]= 6 Adj[2,7]= - Adj[2,8]= -
Adj[3,0]= - Adj[3,1]= - Adj[3,2]= 5 Adj[3,3]= - Adj[3,4]= 3 Adj[3,5]= - Adj[3,6]= - Adj[3,7]= - Adj[3,8]= -
Adj[4,0]= - Adj[4,1]= - Adj[4,2]= - Adj[4,3]= 3 Adj[4,4]= - Adj[4,5]= - Adj[4,6]= 2 Adj[4,7]= 4 Adj[4,8]= -
Adj[5,0]= 5 Adj[5,1]= 3 Adj[5,2]= - Adj[5,3]= - Adj[5,4]= - Adj[5,5]= - Adj[5,6]= 7 Adj[5,7]= - Adj[5,8]= 5
Adj[6,0]= - Adj[6,1]= - Adj[6,2]= 6 Adj[6,3]= - Adj[6,4]= 2 Adj[6,5]= 7 Adj[6,6]= - Adj[6,7]= 6 Adj[6,8]= -
Adj[7,0]= - Adj[7,1]= - Adj[7,2]= - Adj[7,3]= - Adj[7,4]= 4 Adj[7,5]= - Adj[7,6]= 6 Adj[7,7]= - Adj[7,8]= 8
Adj[8,0]= - Adj[8,1]= - Adj[8,2]= - Adj[8,3]= - Adj[8,4]= - Adj[8,5]= 5 Adj[8,6]= - Adj[8,7]= 8 Adj[8,8]= -

fim da impressao do grafo.

Prim a partir de 1:
  1 - 2  (peso 4)
  2 - 3  (peso 7)
  3 - 4  (peso 5)
  4 - 5  (peso 3)
  2 - 6  (peso 3)
  5 - 7  (peso 2)
  5 - 8  (peso 4)
  6 - 9  (peso 5)
  Peso total da AGM: 33
  Esperado: 33 -> OK
  Peso 33 a partir de qualquer origem: OK
```

```
=== GRAFO 2 (Atividade Anterior) ===
n: 6
m: 9

Adj[0,0]= - Adj[0,1]= 6 Adj[0,2]= 15 Adj[0,3]= - Adj[0,4]= 8 Adj[0,5]= -
Adj[1,0]= 6 Adj[1,1]= - Adj[1,2]= 20 Adj[1,3]= - Adj[1,4]= 10 Adj[1,5]= 8
Adj[2,0]= 15 Adj[2,1]= 20 Adj[2,2]= - Adj[2,3]= 9 Adj[2,4]= - Adj[2,5]= -
Adj[3,0]= - Adj[3,1]= - Adj[3,2]= 9 Adj[3,3]= - Adj[3,4]= - Adj[3,5]= 7
Adj[4,0]= 8 Adj[4,1]= 10 Adj[4,2]= - Adj[4,3]= - Adj[4,4]= - Adj[4,5]= 5
Adj[5,0]= - Adj[5,1]= 8 Adj[5,2]= - Adj[5,3]= 7 Adj[5,4]= 5 Adj[5,5]= -

fim da impressao do grafo.

Prim a partir de a:
  a - b  (peso 6)
  d - c  (peso 9)
  f - d  (peso 7)
  a - e  (peso 8)
  e - f  (peso 5)
  Peso total da AGM: 35
  Esperado: 35 -> OK
```

```
=== EXTRA: grafo desconexo ===
Prim a partir de A:
  A - B  (peso 1)
  Grafo desconexo: AGM não existe (floresta com 1 arestas, peso 1).
```

UNIVERSIDADE PRESBITERIANA MACKENZIE
Faculdade de Computação e Informática
Prof. Dr. Ivan Carlos Alcântara de Oliveira
Disciplina: Teoría dos Grafos

Apêndice

Link do GitHub:
https://github.com/M4t3u5534/implementacao-prim.git